OOP LAB — SESSION 4

Structures & Nested Structures in C++

Superior University Lahore · Dept. of Robotics & AI

Student: Mubashar Ali · ID: SU91-BSRAM-F25-043

Programming Fundamentals · Ms. Eisha Nawaz

Task	Description
Lab Task 1	Nested Struct: Student Record (Result + Record)
Lab Task 2	Nested Struct: Car Details (Data + Car)
Lab Task 3	Complex Number Calculator (Add / Subtract / Multiply)
Lab Task 4	Library System: Books Published After 2015 (Author + Book)
Lab Task 5	Student GPA Calculator (struct + function)
Lab Task 6	Bank Account System (deposit / withdraw)
Home Task 1	Phonebook Entry with Date (Date + Phonebook)
Home Task 2	Rectangle Area & Perimeter (Parameters + Result + Rectangle)
Home Task 3	Course Enrollment System (Instructor + Course)
Home Task 4	Hospital Billing System (Date + Doctor + Patient)

■ LAB TASK 1 — Student Record (Nested Structs)

■ Concept: Struct inside struct: Result nested in Record. Access via dot-chain r.res.marks

■ CODE

```
#include
using namespace std;

struct Result {
    float marks;
    char  grade;
};

struct Record {
    int    rollNo;
    Result res;
};

int main() {
    Record r;
    cout << "Enter Roll Number: ";
    cin  >> r.rollNo;
    cout << "Enter Marks: ";
    cin  >> r.res.marks;
    cout << "Enter Grade: ";
    cin  >> r.res.grade;

    cout << "\n--- Student Record ---\n";
    cout << "Roll Number: " << r.rollNo    << "\n";
    cout << "Marks: "       << r.res.marks  << "\n";
    cout << "Grade: "       << r.res.grade  << "\n";
    return 0;
}
```

■ OUTPUT

```
Enter Roll Number: 23454
Enter Marks: 900
Enter Grade: A

--- Student Record ---
Roll Number: 23454
Marks: 900
Grade: A

Process finished with exit code 0
```

■ LAB TASK 2 — Car Details (Nested Structs)

■ Concept: Struct Data (name, color) nested inside struct Car — demonstrates multi-level member access

■ CODE

```
#include
#include
using namespace std;

struct Data {
    string name;
    string color;
};

struct Car {
    string model;
    float  topSpeed;
    int    noOfGears;
    Data   d;
};

int main() {
    Car c;
    cout << "Enter Car Name: ";   cin >> c.d.name;
    cout << "Enter Car Color: ";  cin >> c.d.color;
    cout << "Enter Model: ";      cin >> c.model;
    cout << "Enter Top Speed: ";  cin >> c.topSpeed;
    cout << "Enter No of Gears: ";cin >> c.noOfGears;

    cout << "\n--- Car Details ---\n";
    cout << "Name: "     << c.d.name     << "\n";
    cout << "Color: "    << c.d.color    << "\n";
    cout << "Model: "    << c.model      << "\n";
    cout << "Top Speed: "<< c.topSpeed   << "\n";
    cout << "Gears: "    << c.noOfGears  << "\n";
    return 0;
}
```

■ OUTPUT

```
--- Car Details ---
Name: FERRARI
Color: RED
Model: 2022
Top Speed: 300
Gears: 5

Process finished with exit code 0
```

■ LAB TASK 3 — Complex Number Calculator

■ Concept: Struct used as function parameter & return type. Implements add, subtract, multiply on complex numbers.

■ CODE

```
#include
using namespace std;

struct Complex {
    float real;
    float imag;
};

Complex addition(Complex a, Complex b) {
    return {a.real + b.real, a.imag + b.imag};
}

Complex subtract(Complex a, Complex b) {
    return {a.real - b.real, a.imag - b.imag};
}

Complex multiply(Complex a, Complex b) {
    // (a+bi)(c+di) = (ac-bd) + (ad+bc)i
    return {a.real*b.real - a.imag*b.imag,
            a.real*b.imag + a.imag*b.real};
}

int main() {
    Complex num1, num2, ans;
    cout << "Enter real part of first number: ";  cin >> num1.real;
    cout << "Enter imaginary part of first: ";    cin >> num1.imag;
    cout << "Enter real part of second number: "; cin >> num2.real;
    cout << "Enter imaginary part of second: ";   cin >> num2.imag;

    cout << "\n--- Results ---\n";

    ans = addition(num1, num2);
    cout << "Sum: "        << ans.real << " + " << ans.imag << "i\n";

    ans = subtract(num1, num2);
    cout << "Difference: " << ans.real << " + " << ans.imag << "i\n";

    ans = multiply(num1, num2);
    cout << "Product: "    << ans.real << " + " << ans.imag << "i\n";
    return 0;
}
```

■ OUTPUT

```
--- Results ---
Sum: 79 + 45i
Difference: 33 + 23i
Product: 914 + 1398i

Process finished with exit code 0
```

■ LAB TASK 4 — Library System (Array of Structs)

■ Concept: Array of struct Book, each containing a nested Author struct. Filter loop prints books published after 2015.

■ CODE

```
#include
#include
using namespace std;

struct Author {
    string name;
    string nationality;
};

struct Book {
    string title;
    string ISBN;
    double price;
    int    pubYear;
    Author author;
};

int main() {
    Book library[3];

    // Hardcoded sample data
    library[0] = {"UNDER THE SHADE", "978-0", 450.0, 2018, {"H.J.Mike","UK"}};
    library[1] = {"OLD PATHS",        "978-1", 320.0, 2010, {"T.A.James","US"}};
    library[2] = {"Bricks And Peace", "978-2", 600.0, 2022, {"W.J.Kirk","AU"}};

    cout << "Books published after 2015:\n";
    for (int i = 0; i < 3; i++) {
        if (library[i].pubYear > 2015) {
            cout << "Title: "  << library[i].title
                 << " | Year: "<< library[i].pubYear
                 << " | Author: " << library[i].author.name << "\n";
        }
    }
    return 0;
}
```

■ OUTPUT

```
Books published after 2015:
Title: UNDER THE SHADE | Year: 2018 | Author: H.J.Mike
Title: Bricks And Peace | Year: 2022 | Author: W.J.Kirk

Process finished with exit code 0
```

■ LAB TASK 5 — Student GPA Calculator

■ Concept: Pass-by-reference function modifies gpa field inside struct. GPA capped at 4.0. Two students entered at runtime.

■ CODE

```
#include
#include
using namespace std;

struct Student {
    string name;
    string rollNo;
    float  marks[3];
    float  gpa;
};

void calculateGPA(Student &s;) {
    s.gpa = (s.marks[0] + s.marks[1] + s.marks[2]) / 3.0f / 10.0f;
    if (s.gpa > 4.0f) s.gpa = 4.0f;
}

void printStudent(const Student &s;) {
    cout << "Name    : " << s.name   << "\n";
    cout << "Roll No : " << s.rollNo << "\n";
    cout << "Marks   : " << s.marks[0] << " " << s.marks[1] << " " << s.marks[2] << "\n";
    cout << "GPA     : " << s.gpa << " / 4.0\n";
}

int main() {
    Student students[2];

    // Student 1 — hardcoded
    students[0] = {"MUBASHAR ALI", "4353", {97, 98, 96}, 0};
    calculateGPA(students[0]);
    cout << "Student Details:\n";
    printStudent(students[0]);

    // Student 2 — user input
    cout << "\n--- Student 2 ---\n";
    cout << "Enter name: ";        cin >> students[1].name;
    cout << "Enter roll number: "; cin >> students[1].rollNo;
    for (int i = 0; i < 3; i++) {
        cout << "Enter marks for Subject " << i+1 << ": ";
        cin  >> students[1].marks[i];
    }
    calculateGPA(students[1]);
    printStudent(students[1]);
    return 0;
}
```

■ OUTPUT

```
Student Details:
Name    : MUBASHAR ALI
Roll No : 4353
Marks   : 97 98 96
GPA     : 4 / 4.0

--- Student 2 ---
Enter name: KHADIJA
Enter roll number: 96
Enter marks for Subject 1: 95
Enter marks for Subject 2: 98
Enter marks for Subject 3: (continues...)
```

■ LAB TASK 6 — Bank Account System

■ Concept: Struct returned from function (createAccount). Separate deposit/withdraw logic with balance guard.

■ CODE

```
#include
#include
using namespace std;

struct Account {
    string accountNumber;
    string holderName;
    double balance;
};

Account createAccount() {
    Account acc;
    cout << "=== Create New Account ===\n";
    cout << "Enter account number: ";
    getline(cin >> ws, acc.accountNumber);
    cout << "Enter account holder name: ";
    getline(cin >> ws, acc.holderName);
    cout << "Enter opening balance: ";
    cin  >> acc.balance;
    return acc;
}

void deposit(Account &a;, double amount) {
    a.balance += amount;
    cout << "Deposit successful! Balance: Rs. " << a.balance << "\n";
}

bool withdraw(Account &a;, double amount) {
    if (amount > a.balance) {
        cout << "Insufficient funds!\n";
        return false;
    }
    a.balance -= amount;
    cout << "Withdrawal successful!\n";
    cout << "Balance after withdrawal: Rs. " << a.balance << "\n";
    return true;
}

int main() {
    Account acc = createAccount();
    double amt;
    for (int i = 1; i <= 2; i++) {
        cout << "Enter amount for withdrawal " << i << ": ";
        cin  >> amt;
        withdraw(acc, amt);
    }
    cout << "\nFinal Balance: Rs. " << acc.balance << "\n";
    return 0;
}
```

■ OUTPUT

```
Enter amount for withdrawal 1: 2340
Withdrawal successful!
Balance after withdrawal: Rs. 284660

Enter amount for withdrawal 2: 1000
Withdrawal successful!
Balance after withdrawal: Rs. 283660

Final Balance: Rs. 283660

Process finished with exit code 0
```

■ HOME TASK 1 — Phonebook Entry with Date

■ Concept: Date struct nested inside Phonebook struct. Stores contact info with day/month/year entry date.

■ CODE

```
#include
#include
using namespace std;

struct Date {
    int day, month, year;
};

struct Phonebook {
    string name;
    string city;
    string phoneNumber;
    Date   d;
};

int main() {
    Phonebook pb;
    cout << "Enter Name: ";         cin >> pb.name;
    cout << "Enter City: ";         cin >> pb.city;
    cout << "Enter Phone Number: "; cin >> pb.phoneNumber;
    cout << "Enter Date (Day Month Year): ";
    cin  >> pb.d.day >> pb.d.month >> pb.d.year;

    cout << "\n--- Phonebook Record ---\n";
    cout << "Name:  "  << pb.name        << "\n";
    cout << "City:  "  << pb.city        << "\n";
    cout << "Phone: "  << pb.phoneNumber << "\n";
    cout << "Date:  "  << pb.d.day << "/"
                       << pb.d.month << "/"
                       << pb.d.year  << "\n";
    return 0;
}
```

■ OUTPUT

```
Enter Date (Day Month Year): 15
01
2004

--- Phonebook Record ---
Name:  ALI
City:  LAHORE
Phone: 43564875638
Date:  15/1/2004

Process finished with exit code 0
```

■ HOME TASK 2 — Rectangle Area & Perimeter

■ Concept: Three-struct design: Parameters (length, width), Result (area, perimeter), Rectangle wraps both.

■ CODE

```
#include
using namespace std;

struct Parameters {
    float length;
    float width;
};

struct Result {
    float area;
    float perimeter;
};

struct Rectangle {
    Parameters p;
    Result     r;
};

int main() {
    Rectangle rect;
    cout << "Enter Length: "; cin >> rect.p.length;
    cout << "Enter Width: ";  cin >> rect.p.width;

    rect.r.area      = rect.p.length * rect.p.width;
    rect.r.perimeter = 2 * (rect.p.length + rect.p.width);

    cout << "\nArea: "      << rect.r.area
         << "\nPerimeter: " << rect.r.perimeter << "\n";
    return 0;
}
```

■ OUTPUT

```
Enter Length: 12
Enter Width: 13

Area: 156
Perimeter: 50

Process finished with exit code 0
```

■ HOME TASK 3 — Course Enrollment System

■ Concept: Instructor struct nested in Course. Tracks maxSeats vs enrolledStudents. Rejects enrollment when full.

■ CODE

```
#include
#include
using namespace std;

struct Instructor {
    string name;
    string department;
};

struct Course {
    string     courseCode;
    string     courseName;
    int        creditHours;
    int        maxSeats;
    int        enrolledStudents;
    Instructor instructor;
};

bool enroll(Course &c;, int studentId) {
    if (c.enrolledStudents >= c.maxSeats) {
        cout << "Enrolling student " << studentId
             << " in " << c.courseCode << "... Full!\n";
        return false;
    }
    c.enrolledStudents++;
    cout << "Enrolling student " << studentId
         << " in " << c.courseCode << "... Success!\n";
    return true;
}

void printCourse(const Course &c;) {
    int remaining = c.maxSeats - c.enrolledStudents;
    cout << "=== Final Course Details ===\n";
    cout << "Code         : " << c.courseCode   << "\n";
    cout << "Name         : " << c.courseName   << "\n";
    cout << "Credit Hours : " << c.creditHours  << "\n";
    cout << "Instructor   : " << c.instructor.name
         << " (" << c.instructor.department << ")\n";
    cout << "Seats        : " << c.enrolledStudents
         << " / " << c.maxSeats
         << " (" << remaining << " remaining)\n";
}

int main() {
    Course c1 = {"PYHTON","XX239",17,19,0,{"MISS JAVERIA","FET"}};
    Course c2 = {"4345","OOP",3,30,0,{"MS EISHA","FET"}};

    for (int i = 1; i <= 13; i++) enroll(c1, i);
    printCourse(c1);
    printCourse(c2);
    return 0;
}
```

■ OUTPUT

```
Enrolling student 11 in 4345... Success!
Enrolling student 12 in 4345... Success!

=== Final Course Details ===
Code         : PYHTON
Name         : XX239
Credit Hours : 17
Instructor   : MISS JAVERIA (FET)
Seats        : 13 / 19 (6 remaining)

Code         : 4345
Name         : OOP
Credit Hours : 3   (continues...)
```

■ HOME TASK 4 — Hospital Billing System

■ Concept: Three-level nesting: Date + Doctor inside Patient. Total bill = daysAdmitted x dailyCharge. Runtime input.

■ CODE

```
#include
#include
using namespace std;

struct Date {
    int day, month, year;
};

struct Doctor {
    string name;
    string specialization;
};

struct Patient {
    int    patientId;
    string name;
    int    age;
    Date   admissionDate;
    Doctor doctor;
    int    daysAdmitted;
    double dailyCharge;
};

double calculateBill(const Patient &p;) {
    return p.daysAdmitted * p.dailyCharge;
}

int main() {
    Patient p;
    cout << "Enter Patient ID: ";      cin >> p.patientId;
    cout << "Enter Name: ";            cin >> p.name;
    cout << "Enter Age: ";             cin >> p.age;
    cout << "Enter Admission (D M Y): ";
    cin  >> p.admissionDate.day >> p.admissionDate.month >> p.admissionDate.year;
    cout << "Enter Doctor Name: ";     cin >> ws; getline(cin, p.doctor.name);
    cout << "Enter Specialization: ";  getline(cin, p.doctor.specialization);
    cout << "Enter Days Admitted: ";   cin >> p.daysAdmitted;
    cout << "Enter Daily Charge: ";    cin >> p.dailyCharge;

    double bill = calculateBill(p);

    cout << "\nPatient ID     : " << p.patientId << "\n";
    cout << "Name           : " << p.name       << "\n";
    cout << "Age            : " << p.age        << "\n";
    cout << "Admission      : " << p.admissionDate.day << " / "
                               << p.admissionDate.month << " / "
                               << p.admissionDate.year  << "\n";
    cout << "Doctor         : " << p.doctor.name           << "\n";
    cout << "Specialization : " << p.doctor.specialization  << "\n";
    cout << string(30,'-') << "\n";
    cout << "Days Admitted  : " << p.daysAdmitted  << "\n";
    cout << "Daily Charge   : Rs. " << p.dailyCharge << "\n";
    cout << "TOTAL BILL     : Rs. " << bill           << "\n";
    cout << string(30,'=') << "\n";
    return 0;
}
```

■ **OUTPUT**

```
Patient ID     : 56
Name           : ALEX
Age            : 21
Admission      : 13 / 2 / 2026
Doctor         : DR RAZIA
Specialization : PYSIOTHERAPY
------------------------------
Days Admitted  : 12
Daily Charge   : Rs. 13000
TOTAL BILL     : Rs. 156000
==============================

Process finished with exit code 0
```